



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

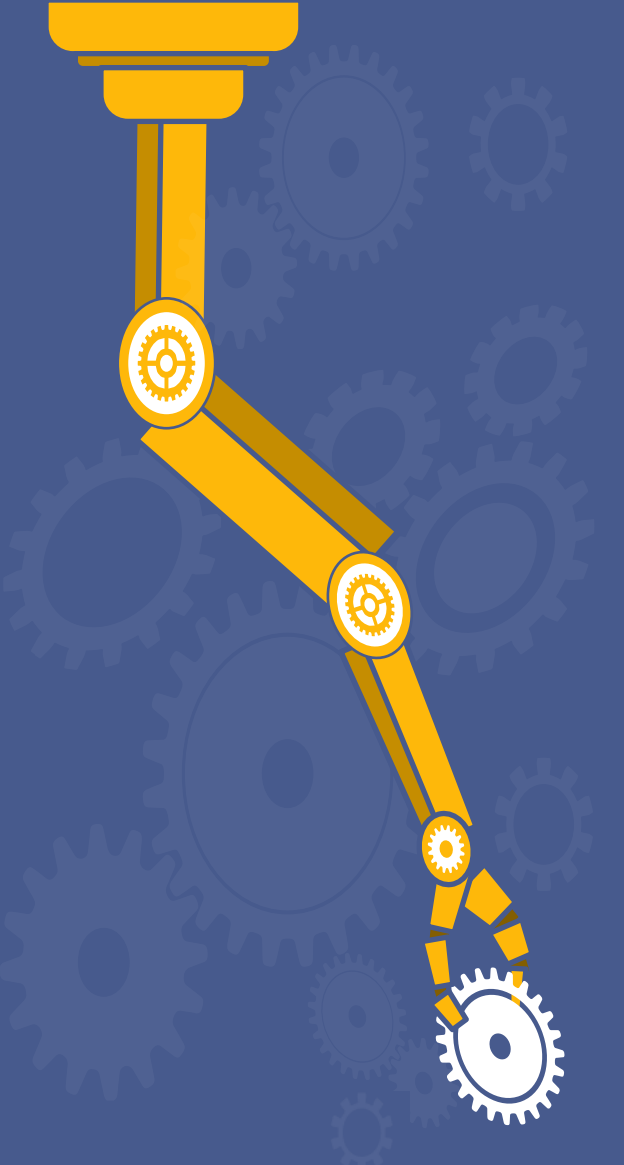
BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms-

Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - <u>Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.</u>		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours



BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos, Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module 5 : Geometric Algorithms (4 Hours)

- Line Segments, Properties, Intersection, Sweeping Lines
- **Convex hull finding Algorithm**
- **Graham's Scan**
- Jarvis March Algorithm



Convex Hull



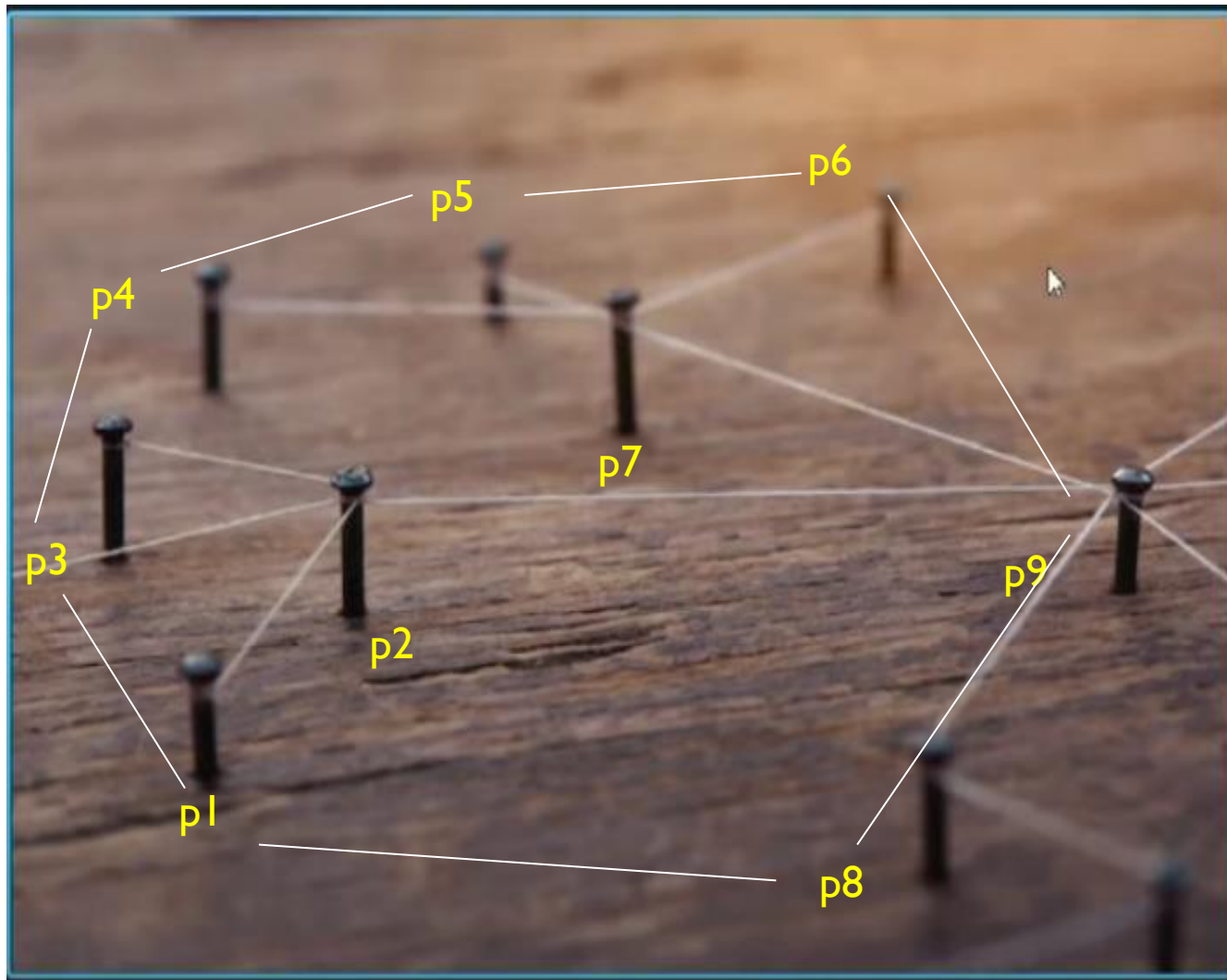
Problem Statement: Having set of points , **our objective is to draw the convex polygon**, How it should be?? - All points should be covered – either inside polygon or in the boundary.

Note: No .of points ≥ 3 and points should not be in collinear





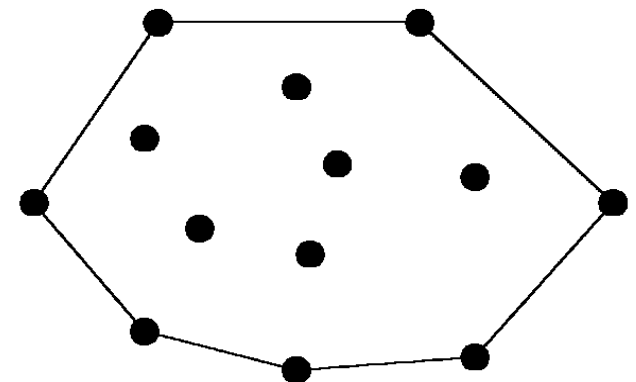
Convex Hull





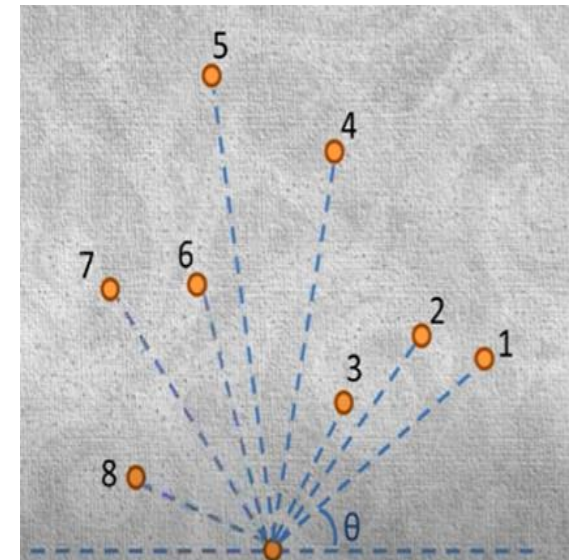
Convex Hull- Definition

- The convex hull of a set Q of points, denoted by $CH(Q)$ is the smallest convex polygon P for which each point in Q is either on the boundary of P or in its interior.
- **Note :** All points in the set Q are unique and that Q contains at least three points which are not colinear.
- Vertices maximizes the area while minimizing the circumference.



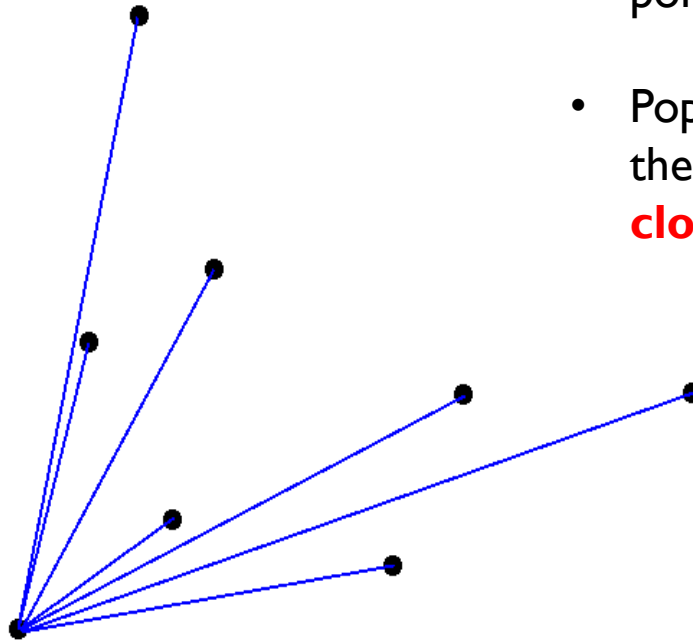
Graham's Scan

- Start at point guaranteed to be on the hull. (the point with the minimum y value i.e lowest y coordinate).
- **Sort** remaining points by **polar angles** of vertices relative to the first point.
- Go through sorted points, keeping vertices of points that have **left turns**(turn related to the previous 2 points) and dropping points that have **right turns**.



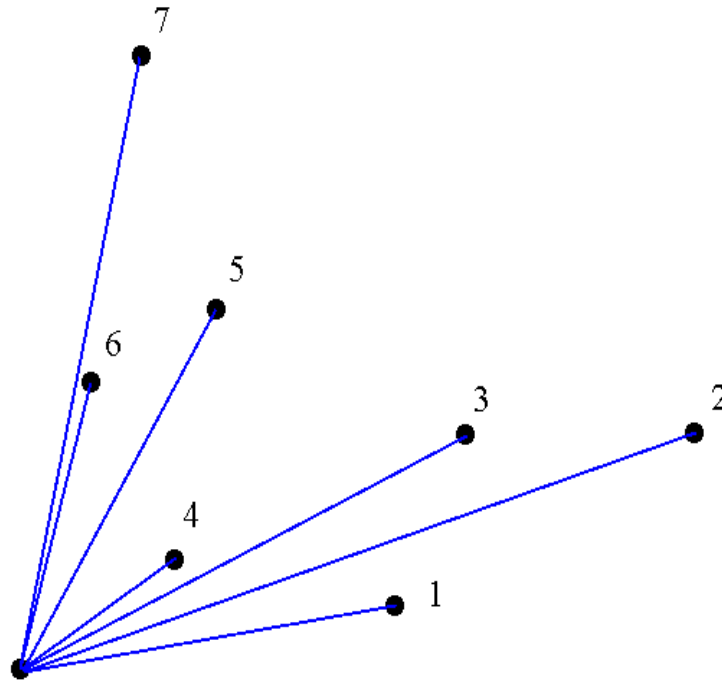
Graham's Scan

- Iterate in the sorted order, placing each point on the stack, but only if it makes a **counter clockwise** turn relative to the previous two points on the stack
- Pop previous point off of the stack if making a **clockwise** turn.



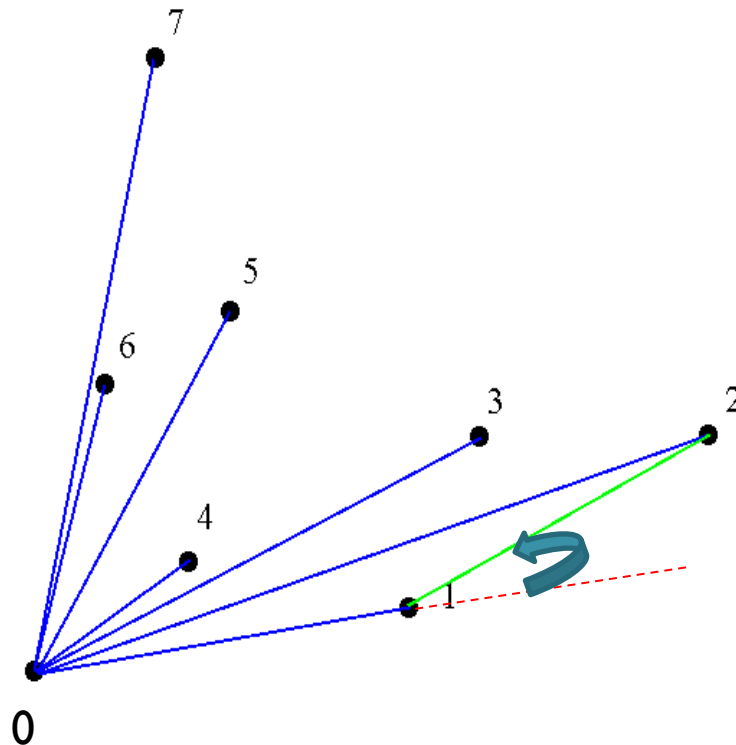


Graham's Scan

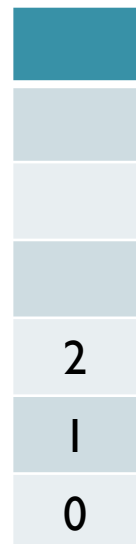




Graham's Scan

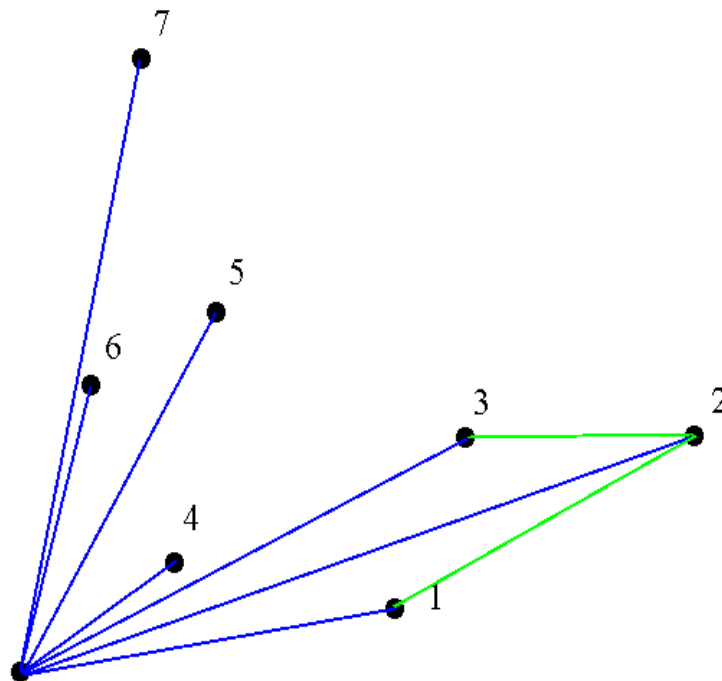


STACK



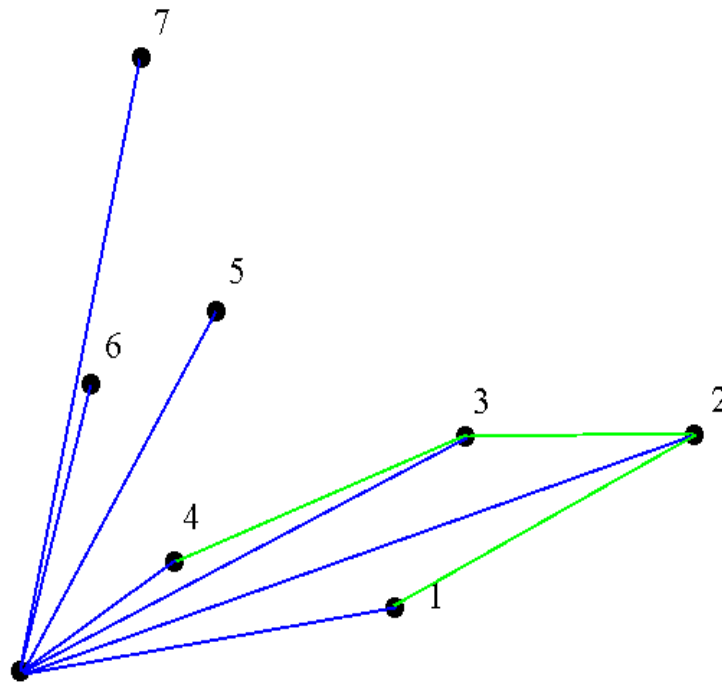


Graham's Scan



3
2
1
0

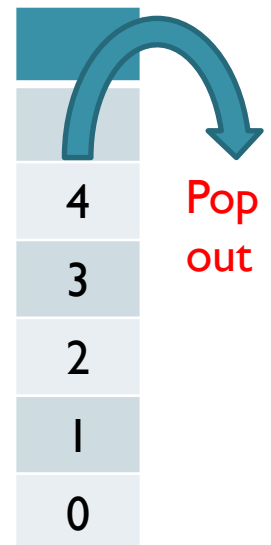
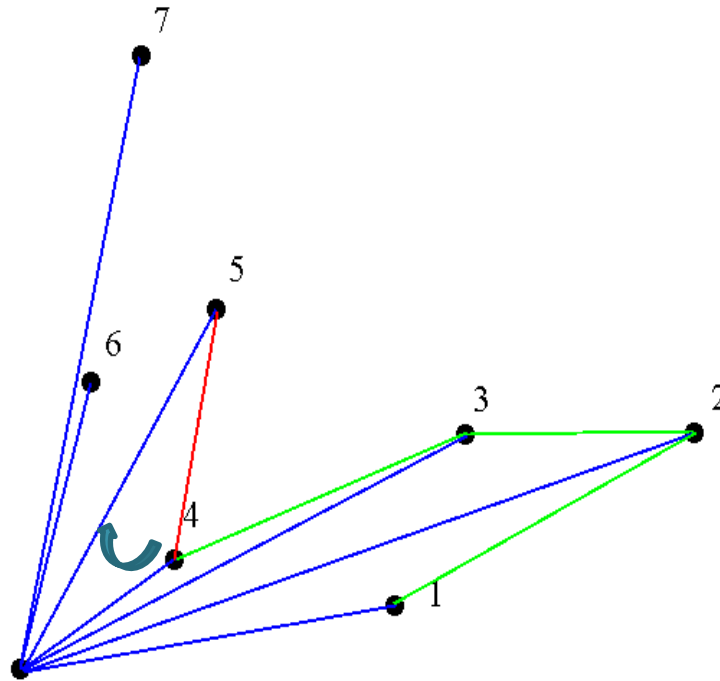
Graham's Scan



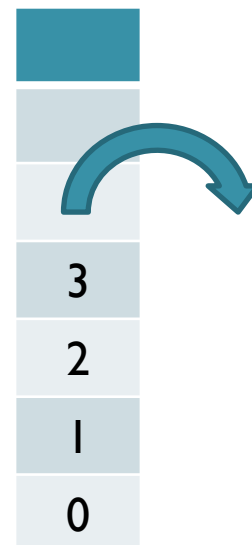
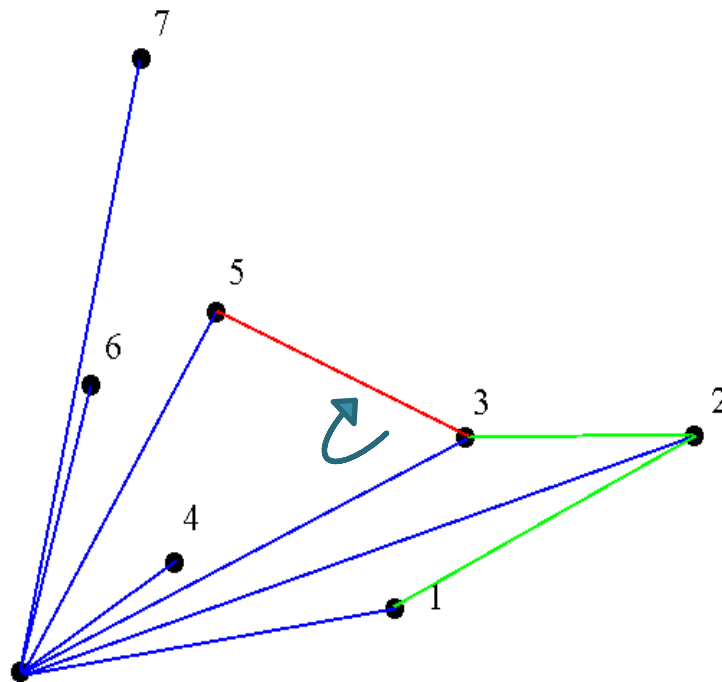
4
3
2
1
0



Graham's Scan

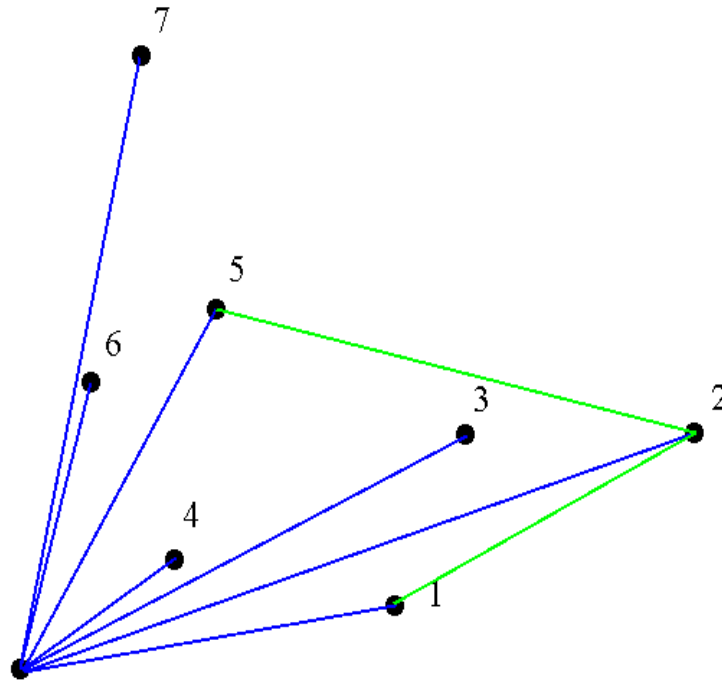


Graham's Scan



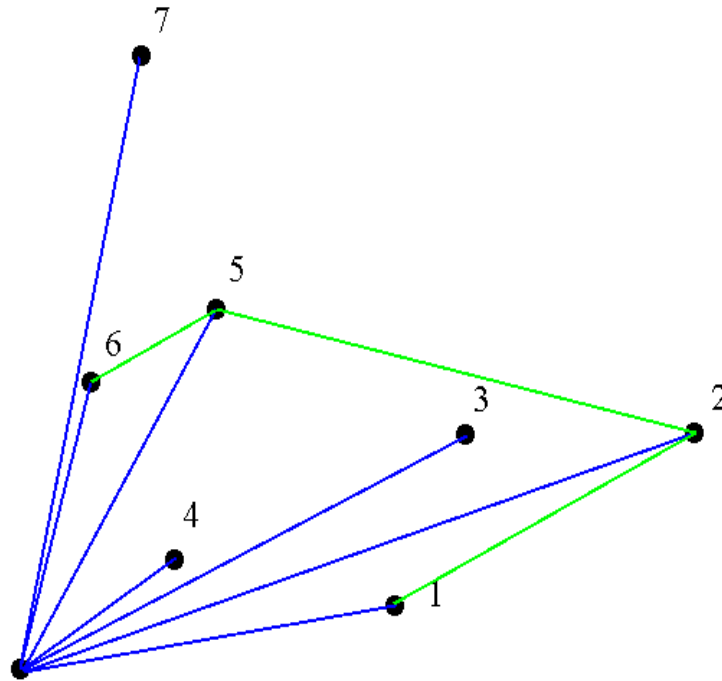


Graham's Scan



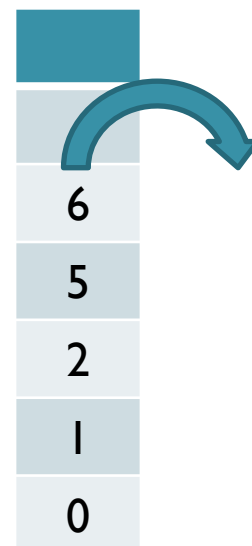
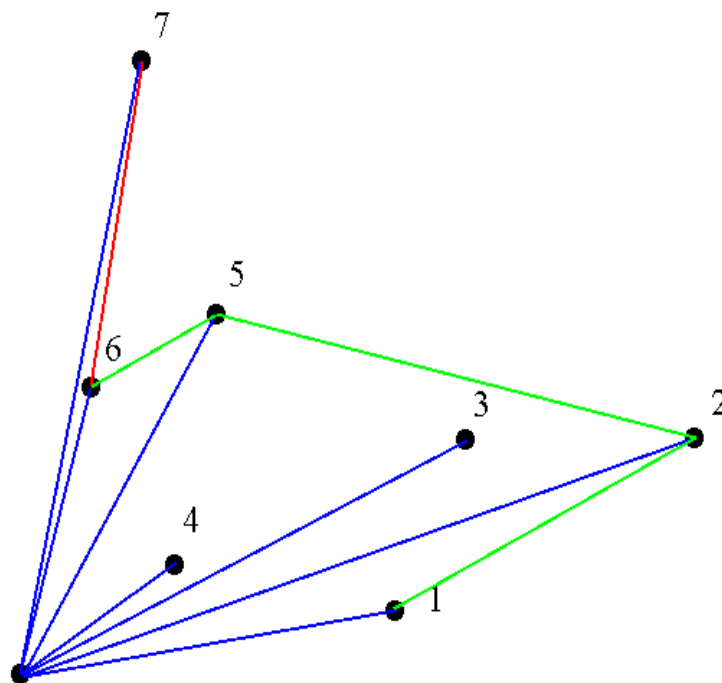
5
2
1
0

Graham's Scan

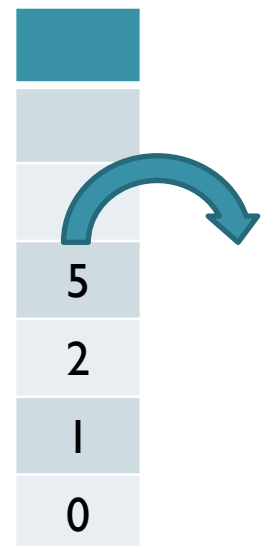
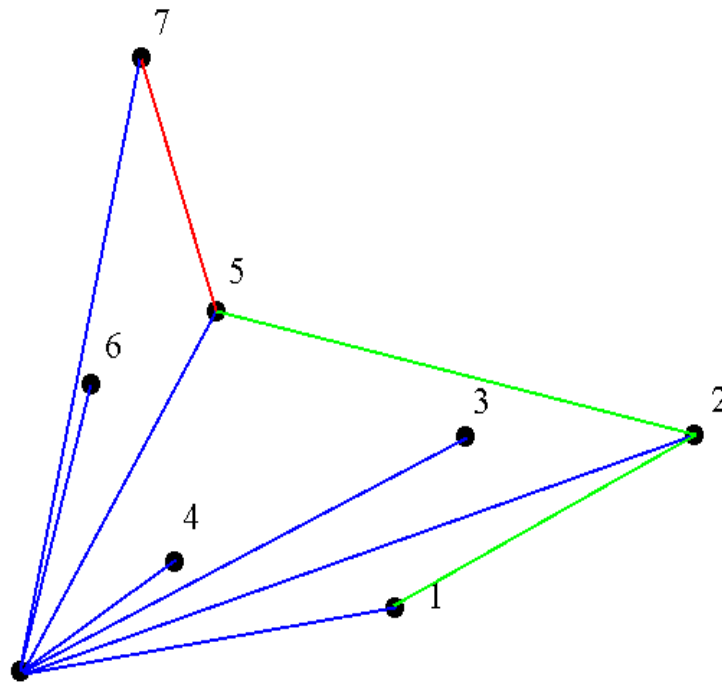


6
5
2
1
0

Graham's Scan

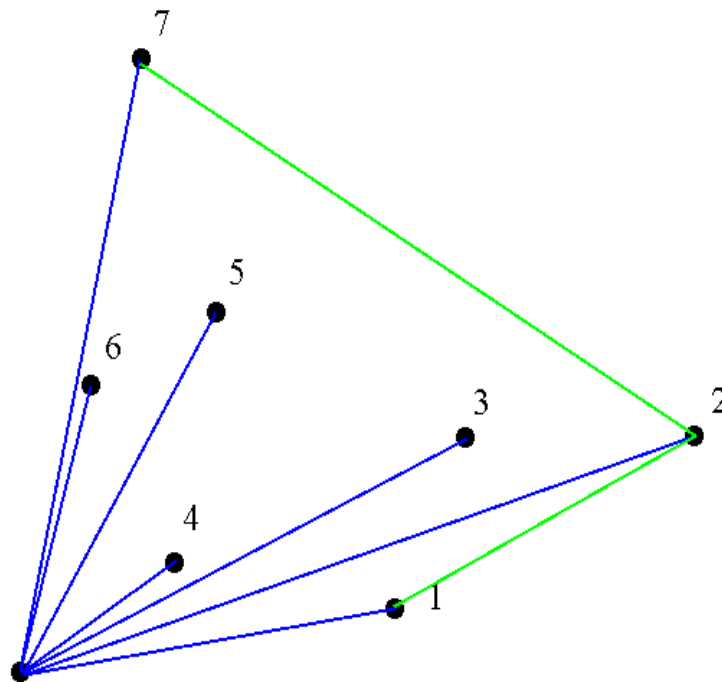


Graham's Scan





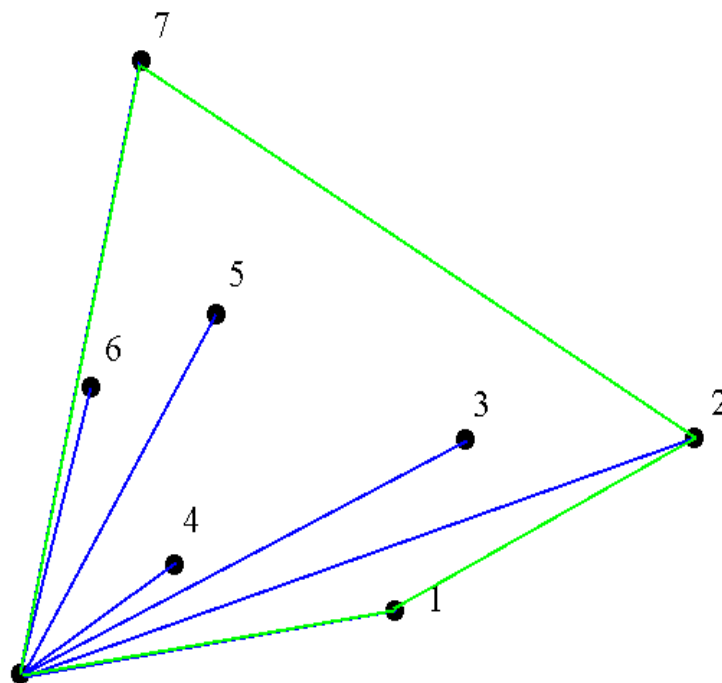
Graham's Scan



2
1
0



Graham's Scan



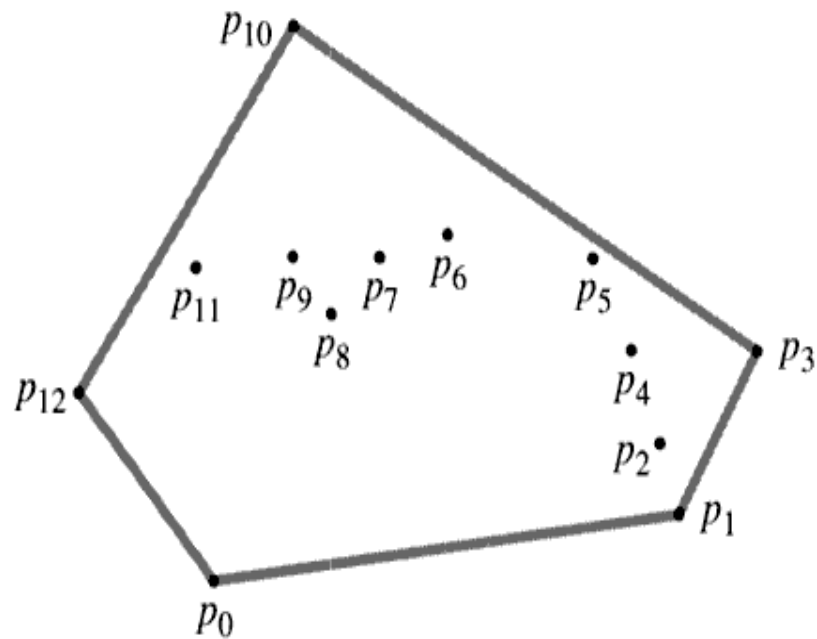


Figure 33.6 A set of points $Q = \{p_0, p_1, \dots, p_{12}\}$ with its convex hull $\text{CH}(Q)$ in gray.

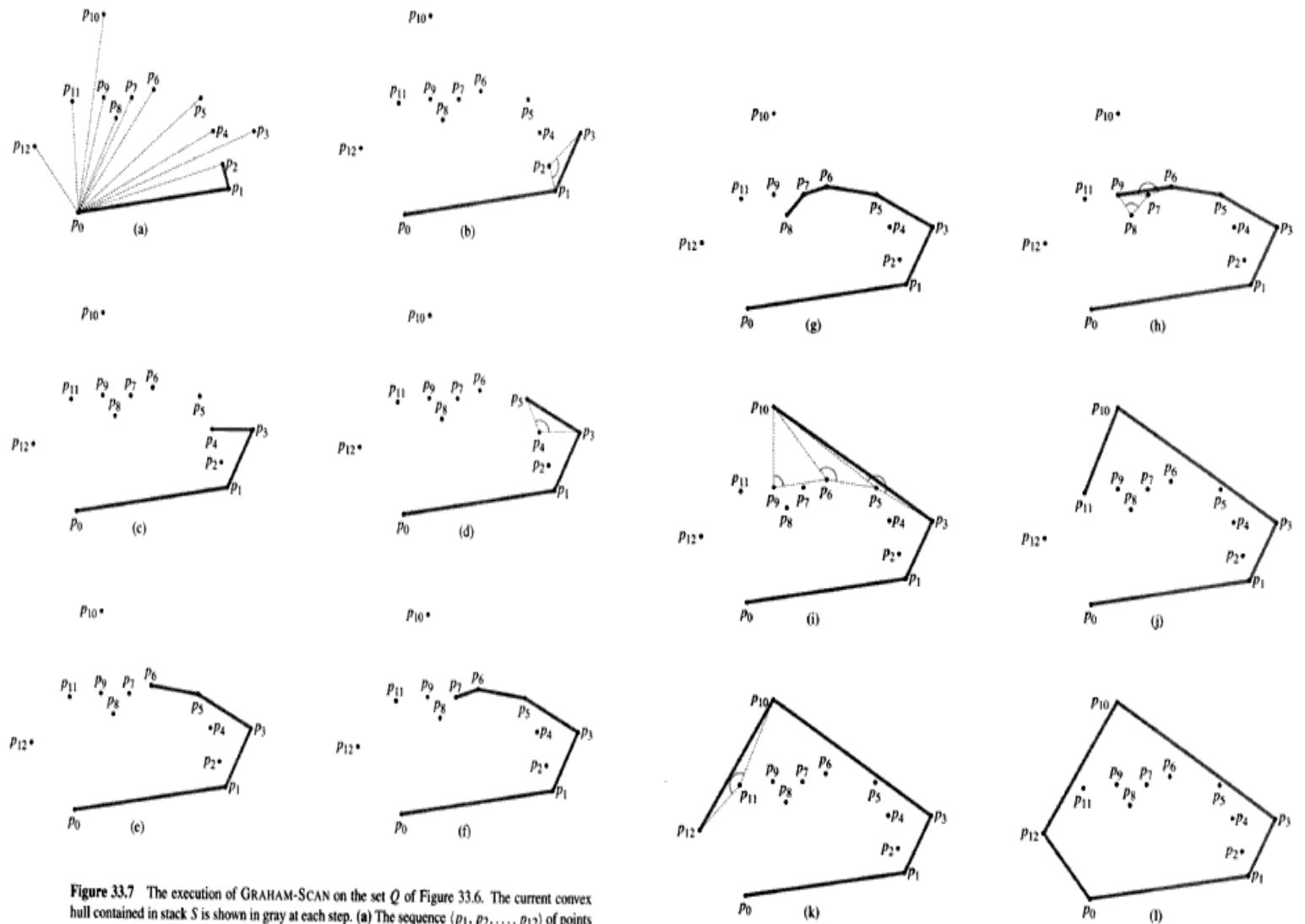


Figure 33.7 The execution of GRAHAM-SCAN on the set Q of Figure 33.6. The current convex hull contained in stack S is shown in gray at each step. (a) The sequence $\langle p_1, p_2, \dots, p_{12} \rangle$ of points numbered in order of increasing polar angle relative to p_0 , and the initial stack S containing p_0, p_1 , and p_2 . (b)–(k) Stack S after each iteration of the **for** loop of lines 6–9. Dashed lines show nonleft turns, which cause points to be popped from the stack. In part (h), for example, the right turn at angle $\angle p_7 p_8 p_9$ causes p_8 to be popped, and then the right turn at angle $\angle p_6 p_7 p_9$ causes p_7 to be popped. (l) The convex hull returned by the procedure, which matches that of Figure 33.6.

Algorithm – Time Analysis

GRAHAM-SCAN(Q)

- 1 let p_0 be the point in Q with the minimum y-coordinate,
or the leftmost such point in case of a tie
- 2 let $\langle p_1, p_2, \dots, p_m \rangle$ be the remaining points in Q ,
sorted by polar angle in counterclockwise order around p_0
(if more than one point has the same angle, remove all but
the one that is farthest from p_0)
- 3 PUSH(p_0, S)
- 4 PUSH(p_1, S)
- 5 PUSH(p_2, S)
- 6 for $i \leftarrow 3$ to m
- 7 do while the angle formed by points NEXT-TO-TOP(S), TOP(S),
 and p_i makes a nonleft turn
- 8 do POP(S)
- 9 PUSH(p_i, S)
- 10 return S

**$O(1)$ time for all
push operation**

$N - O(n)$

**$O(1)$ time for
finding polar
angle
Sorting of n
points- ($n \log n$)**

$O(1)$

**$O(n)$
 n - push
operations
 $m \leq n-1$ - pop
operations**

Overall - $O(n \log n)$





Thank you